

The `ftxc` Command-Line Compiler — User Manual

FlashTeX Documentation Team

January 1, 1970

1 Introduction

This manual documents `ftxc`, a small command-line utility that wraps the FlashTeX rendering engine for batch document conversion. It is intended for users who are comfortable invoking tools such as `pdflatex` or `make` from a terminal. The tool is distributed as a single static binary and requires no additional runtime dependencies once installed.

Throughout this manual we use `monospace` type to denote literal commands, file names such as `main.tex` or `Makefile`, and configuration keys like `output.format`. Inline snippets of code are written with `ftxc build --watch` when a full listing would be excessive.

For the latest release notes see <https://example.org/ftxc/changelog> or browse the project homepage at [the ftxc project page](https://example.org/ftxc/). Bug reports should be filed at <https://example.org/ftxc/issues> rather than emailed directly.

2 Installation

The recommended installation path is to download a prebuilt release archive and place the `ftxc` binary somewhere on your `PATH`, such as `/usr/local/bin` or `~/.local/bin`. Package maintainers may instead prefer to build from source using the accompanying `Makefile`.

A minimal build from source looks like the transcript in Listing 1, which invokes the small bootstrap compiler written in C that produces the final Rust-based toolchain.

Listing 1: Bootstrap build driver (C)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 /* Very small driver that shells out to the real build system
5    and reports a friendly error if the toolchain is missing. */
6 int main(int argc, char **argv) {
7     const char *target = (argc > 1) ? argv[1] : "release";
8     char cmd[256];
9     int n = snprintf(cmd, sizeof(cmd), "cargo build --%s", target);
10    if (n < 0 || n >= (int)sizeof(cmd)) {
11        fprintf(stderr, "error: target name too long\n");
12        return 1;
13    }
14    return system(cmd);
15 }
```

Once installed, verify the toolchain with `ftxc --version` and confirm that the reported version matches the release you downloaded.

3 Configuration

`ftxc` reads a project-local configuration file named `ftxc.toml`. A typical configuration enables incremental caching and selects an output format, as shown in the short Python helper script in Listing 2, which some users run to generate a starter file programmatically instead of writing `ftxc.toml` by hand.

Listing 2: Generating a starter `ftxc.toml`

```
1 import tomllib
2 import pathlib
3
4 DEFAULT_CONFIG = """
5 [build]
6 incremental = true
7 output = "pdf"
8
9 [watch]
10 debounce_ms = 150
11 ignore = [".git", "target"]
12 """
13
14 def write_default_config(path: pathlib.Path) -> None:
15     if path.exists():
16         raise FileExistsError(f"{path} already exists")
17     path.write_text(DEFAULT_CONFIG.strip() + "\n")
18     # Sanity check: make sure what we just wrote actually parses.
19     with path.open("rb") as fh:
20         tomllib.load(fh)
21
22 if __name__ == "__main__":
23     write_default_config(pathlib.Path("ftxc.toml"))
24     print("wrote ftxc.toml")
```

Configuration keys may also be overridden on the command line, for example `ftxc build --set watch.debounce_ms=50`. Command-line overrides always take precedence over the values found in `ftxc.toml`.

4 Everyday Usage

The typical development loop consists of editing a `.tex` source file, letting `ftxc` rebuild it, and inspecting the resulting PDF. The following steps describe that loop:

- Run `ftxc build` once to produce an initial `main.pdf`.
- Run `ftxc watch` to rebuild automatically whenever `main.tex` or any file it `\inputs` changes.
- Use `ftxc diagnose` to print a structured report of warnings, including overfull and underfull box warnings by page.
- Use `ftxc clean` to remove `.aux`, `.log`, and `.out` artifacts from the working directory.

A short interactive session, including the raw `verbatim` output printed by the tool, is reproduced below exactly as it appears in a terminal:

```
$ ftxc build
[ftxc] reading ftxc.toml
[ftxc] compiling main.tex (pass 1/2)
[ftxc] compiling main.tex (pass 2/2)
[ftxc] wrote main.pdf (3 pages, 0 overfull, 1 underfull)
$ ftxc diagnose --page 2
[ftxc] page 2: underfull \hbox (badness 4021) in paragraph at lines 40--44
```

5 Command-Line Flags

The complete set of top-level flags accepted by `ftxc` is listed below. Flags may be combined freely except where noted.

- `--watch` Rebuild automatically on file changes, using the debounce interval from `watch.debounce_ms`.
- `--set key=value` Override a single configuration key for this invocation only; may be repeated.
- `--format fmt` Select an output format, one of `pdf`, `dvi`, or `ps`.
- `--jobs n` Limit parallel compilation to *n* worker threads; defaults to the number of logical CPUs.
- `--quiet` Suppress all output except errors; useful when `ftxc` is invoked from another `Makefile` target.
- `--version` Print the compiler version and exit immediately without reading `ftxc.toml`.

6 Troubleshooting

If `ftxc build` exits with a nonzero status, first check that `ftxc.toml` is valid by running `ftxc config --check`. Most remaining failures fall into one of two categories: a missing input file referenced via `\input` or `\includegraphics`, or a genuine typesetting error reported by the underlying engine. In the latter case the error message will include a source file and line number, much like the diagnostics produced by `pdflatex` itself.

For further help, consult [the online documentation](#) or open an issue with a minimal reproduction attached.